

Taller practico de MongoDB

Kotlin y KMongo

Guia progresiva para trabajar con MongoDB Atlas desde Kotlin

Documento generado a partir de doc/taller_mongodb.md

Proyecto 2526_PRO_U8_mongo

8 de mayo de 2026

Índice

1	Taller práctico de MongoDB con Kotlin y KMongo	3
1.1	Objetivo	3
1.2	Requisitos	3
1.3	0. Configuración del entorno	4
1.3.1	Qué vas a aprender	4
1.3.2	Variables de entorno	4
1.3.3	Configuración en Kotlin	4
1.3.4	Conexión a MongoDB	4
1.4	1. Módulo 1: gestión de bases de datos	4
1.4.1	Qué vas a aprender	4
1.4.2	Listar bases de datos	5
1.4.3	Seleccionar una base de datos	5
1.4.4	Materializar una base de datos	5
1.4.5	Eliminar una base de datos	5
1.5	Ejercicio 1: explorar bases de datos	5
1.5.1	Objetivo	5
1.5.2	Pasos	5
1.5.3	Resultado esperado	5
1.5.4	Solución	6
1.6	2. Módulo 2: gestión de colecciones	6
1.6.1	Qué vas a aprender	6
1.6.2	Listar colecciones	6
1.6.3	Crear una colección explícitamente	6
1.6.4	Crear una colección de forma implícita	6
1.6.5	Renombrar y eliminar colecciones	6
1.7	Ejercicio 2: colecciones de una biblioteca	6
1.7.1	Objetivo	7
1.7.2	Pasos	7
1.7.3	Solución	7
1.8	3. Módulo 3: operaciones CRUD sobre documentos	7
1.8.1	Qué vas a aprender	7
1.8.2	Crear el servicio de productos	8
1.8.3	Insertar documentos	8
1.8.4	Consultar documentos	8
1.8.5	Actualizar documentos	9
1.8.6	Eliminar documentos	9
1.9	Ejercicio 3: catálogo de productos	9
1.9.1	Objetivo	9
1.9.2	Pasos	9
1.9.3	Solución	9
1.10	Ejercicio 4: actualización de inventario	10
1.10.1	Objetivo	10

1.10.2	Pasos	10
1.10.3	Solución	10
1.11	4. Módulo 4: proyecto integrado de biblioteca digital	10
1.11.1	Qué vas a aprender	10
1.11.2	Modelos principales	11
1.11.3	Crear el servicio de biblioteca	11
1.11.4	Operaciones disponibles	12
1.12	Ejercicio 5: biblioteca digital completa	12
1.12.1	Objetivo	12
1.12.2	Pasos	12
1.12.3	Solución orientativa	12
1.13	Pruebas del proyecto	13
1.13.1	Qué se prueba	13
1.13.2	Herramientas usadas	14
1.14	Ejecutar el proyecto	14
1.14.1	Ejecutar las pruebas	14
1.14.2	Ejecutar la aplicación	15
1.15	Resumen de clases principales	15
1.16	Buenas prácticas aplicadas	15

Taller practico de MongoDB con Kotlin y KMongo

Indice

1 Taller práctico de MongoDB con Kotlin y KMongo

1.1 Objetivo

En este taller aprenderás a trabajar con MongoDB Atlas desde una aplicación Kotlin. El recorrido está organizado de forma progresiva:

1. Configurar la conexión a MongoDB sin escribir credenciales en el código.
2. Gestionar bases de datos.
3. Gestionar colecciones.
4. Realizar operaciones CRUD sobre documentos.
5. Construir un pequeño sistema integrado de biblioteca digital.

La implementación de referencia está en:

`src/main/kotlin/org/iesra/tallermongo/`

Las pruebas unitarias están en:

`src/test/kotlin/org/iesra/tallermongo/`

1.2 Requisitos

- JDK 21.
- Gradle con Kotlin DSL.
- Cuenta en MongoDB Atlas.
- Cluster de MongoDB Atlas creado.
- Usuario de base de datos configurado en Atlas.
- IP autorizada en el panel de acceso de Atlas.

El proyecto usa Kotlin y KMongo:

```
implementation("org.litote.kmongo:kmongo:4.11.0")
```

Para las pruebas unitarias se usa Kotest con DescribeSpec y MockK.

1.3 0. Configuración del entorno

1.3.1 Qué vas a aprender

Antes de operar con MongoDB necesitas configurar la aplicación. La URI de conexión contiene credenciales, por eso no debe escribirse directamente en el código fuente.

1.3.2 Variables de entorno

Crea tus variables de entorno antes de ejecutar la aplicación:

```
MONGODB_URI=mongodb+srv://usuario:password@cluster.mongodb.net/  
MONGODB_DATABASE=taller_mongo
```

El repositorio incluye un fichero `.env.example` con el formato esperado, pero no debes guardar credenciales reales en Git.

1.3.3 Configuración en Kotlin

La clase `MongoConfig` lee la configuración desde variables de entorno:

```
val config = MongoConfig.fromEnvironment()
```

La clase valida que la URI y el nombre de la base de datos no estén vacíos. Si no se define `MONGODB_DATABASE`, se usa `taller_mongo` como base de datos por defecto.

1.3.4 Conexión a MongoDB

La clase `MongoConnection` crea un cliente reutilizable con `KMongo`:

```
MongoConnection(config).use { connection ->  
    val database = connection.database()  
    println(database.name)  
}
```

Usar `use` asegura que el cliente se cierre al terminar el bloque.

Recuerda: crear un cliente por cada operación es una mala práctica. En una aplicación real se reutiliza el cliente mientras la aplicación está activa.

1.4 1. Módulo 1: gestión de bases de datos

1.4.1 Qué vas a aprender

MongoDB crea las bases de datos de forma implícita. Acceder a una base de datos no la materializa hasta que insertas el primer documento.

La clase principal de este módulo es `DatabaseManager`.

1.4.2 Listar bases de datos

```
val databaseManager = DatabaseManager(connection.client())  
val names = databaseManager.listDatabaseNames()  
println(names)
```

1.4.3 Seleccionar una base de datos

```
val academia = databaseManager.selectDatabase("academia")  
println(academia.name)
```

En este punto academia puede no aparecer todavía al listar bases de datos, porque aún no tiene datos.

1.4.4 Materializar una base de datos

```
databaseManager.materializeDatabase(databaseName = "academia", collectionName = "prueba")
```

Esta operación inserta un documento temporal en una colección de prueba. A partir de ahí, MongoDB ya puede mostrar la base de datos.

1.4.5 Eliminar una base de datos

```
databaseManager.dropDatabase("academia")
```

Cuidado: eliminar una base de datos borra todas sus colecciones y documentos.

1.5 Ejercicio 1: explorar bases de datos

1.5.1 Objetivo

Comprobar cómo MongoDB crea una base de datos solo cuando contiene datos.

1.5.2 Pasos

1. Lista las bases de datos disponibles.
2. Selecciona una base de datos llamada academia.
3. Comprueba que puede no aparecer todavía en la lista.
4. Materializa la base de datos insertando un documento temporal en prueba.
5. Vuelve a listar las bases de datos.
6. Elimina academia.

1.5.3 Resultado esperado

Debes observar que academia solo aparece tras insertar el primer documento.

1.5.4 Solución

```
val databaseManager = DatabaseManager(connection.client())

println(databaseManager.listDatabaseNames())

databaseManager.selectDatabase("academia")
println("academia" in databaseManager.listDatabaseNames())

databaseManager.materializeDatabase("academia")
println("academia" in databaseManager.listDatabaseNames())

databaseManager.dropDatabase("academia")
```

1.6 2. Módulo 2: gestión de colecciones

1.6.1 Qué vas a aprender

Una colección agrupa documentos relacionados. Se parece a una tabla en bases de datos relacionales, pero MongoDB permite documentos flexibles.

La clase principal de este módulo es CollectionManager.

1.6.2 Listar colecciones

```
val collectionManager = CollectionManager(database)
println(collectionManager.listCollectionNames())
```

1.6.3 Crear una colección explícitamente

```
collectionManager.createCollectionIfMissing("libros")
```

1.6.4 Crear una colección de forma implícita

```
collectionManager.materializeCollection("usuarios")
```

La colección se crea al insertar el primer documento.

1.6.5 Renombrar y eliminar colecciones

```
collectionManager.renameCollection("usuarios", "socios")
collectionManager.dropCollection("socios")
```

1.7 Ejercicio 2: colecciones de una biblioteca

1.7.1 Objetivo

Crear y gestionar colecciones para una biblioteca.

1.7.2 Pasos

1. Selecciona la base de datos biblioteca.
2. Crea las colecciones libros, autores y socios.
3. Lista las colecciones.
4. Renombra socios a usuarios.
5. Elimina autores.
6. Muestra el estado final.

1.7.3 Solución

```
val database = databaseManager.selectDatabase("biblioteca")
val collections = CollectionManager(database)

collections.createCollectionIfMissing("libros")
collections.createCollectionIfMissing("autores")
collections.createCollectionIfMissing("socios")

println(collections.listCollectionNames())

collections.renameCollection("socios", "usuarios")
collections.dropCollection("autores")

println(collections.listCollectionNames())
```

1.8 3. Módulo 3: operaciones CRUD sobre documentos

1.8.1 Qué vas a aprender

En MongoDB los datos se guardan como documentos BSON. En Kotlin trabajaremos con data class para representar esos documentos de forma clara y tipada.

El modelo usado en este módulo es Product:

```
import org.bson.types.ObjectId

data class Product(
    val _id: String = ObjectId().toHexString(),
    val nombre: String,
    val precio: Double,
    val stock: Int,
    val categoria: String,
    val categorias: List<String> = emptyList(),
```



```
    val descuento: Int? = null,  
)
```

Las operaciones se organizan en dos piezas:

- ProductRepository: contrato de acceso a datos.
- MongoProductRepository: implementación con KMongo.
- ProductService: validaciones y coordinación de operaciones.

Esta separación permite probar la lógica sin conectarse a MongoDB real.

1.8.2 Crear el servicio de productos

```
val products = database.getCollection<Product>("productos")  
val productService = ProductService(MongoProductRepository(products))
```

1.8.3 Insertar documentos

```
val product = Product(  
    nombre = "Libro Kotlin",  
    precio = 34.95,  
    stock = 20,  
    categoria = "libros",  
)  
  
productService.register(product)
```

Para insertar varios documentos:

```
productService.registerMany(  
    listOf(  
        Product(nombre = "Smartphone", precio = 499.0, stock = 30, categoria = "electronica"),  
        Product(nombre = "Camiseta", precio = 19.99, stock = 150, categoria = "ropa"),  
        Product(nombre = "Tablet", precio = 299.0, stock = 45, categoria = "electronica"),  
    )  
)
```

1.8.4 Consultar documentos

```
val allProducts = productService.findAll()  
val expensiveProducts = productService.findExpensiveProducts(50.0)
```

Internamente, MongoProductRepository usa filtros tipados de KMongo, por ejemplo:

```
collection.find(Product::precio gt minimumPrice).toList()
```

1.8.5 Actualizar documentos

```
productService.updatePrice("Libro Kotlin", 29.95)
productService.applyDiscountByCategory("electronica", 10)
productService.increaseStockByCategory("libros", 50)
```

También existe una operación de tipo upsert: actualiza si existe y crea si no existe. En MongoProductRepository se implementa con la extensión de KMongo replaceOneWithFilter y replaceUpsert() para trabajar con el documento Product completo y mantener _id como String.

```
productService.upsert(
    Product(nombre = "Auriculares", precio = 89.99, stock = 60, categoria = "electronica")
)
```

1.8.6 Eliminar documentos

```
productService.deleteByName("Auriculares")
```

La implementación también incluye deleteWithoutStock() en el servicio para eliminar productos con stock menor o igual a cero.

1.9 Ejercicio 3: catálogo de productos

1.9.1 Objetivo

Practicar inserción y consulta de documentos con una colección productos.

1.9.2 Pasos

1. Usa la base de datos configurada para el taller.
2. Obtén la colección tipada productos.
3. Inserta un producto individual.
4. Inserta varios productos más.
5. Muestra todos los productos.
6. Muestra los productos con precio superior a 50 €.

1.9.3 Solución

```
val productService = ProductService(
    MongoProductRepository(database.getCollection<Product>("productos"))
)

productService.register(
    Product(nombre = "Libro Kotlin", precio = 34.95, stock = 20, categoria = "libros")
)
```

```
productService.registerMany(  
    listOf(  
        Product(nombre = "Smartphone", precio = 499.0, stock = 30, categoria = "electronica"),  
        Product(nombre = "Camiseta", precio = 19.99, stock = 150, categoria = "ropa"),  
        Product(nombre = "Tablet", precio = 299.0, stock = 45, categoria = "electronica"),  
        Product(nombre = "Zapatillas", precio = 65.0, stock = 80, categoria = "ropa"),  
    )  
)  
  
println(productService.findAll())  
println(productService.findExpensiveProducts(50.0))
```

1.10 Ejercicio 4: actualización de inventario

1.10.1 Objetivo

Aplicar operaciones de actualización sobre productos ya insertados.

1.10.2 Pasos

1. Actualiza el precio de Libro Kotlin a 29.95 €.
2. Añade un descuento del 10 % a la categoría electronica.
3. Incrementa en 50 unidades el stock de la categoría libros.
4. Usa upsert para crear o actualizar Auriculares.
5. Elimina los descuentos de todos los documentos.

1.10.3 Solución

```
productService.updatePrice("Libro Kotlin", 29.95)  
productService.applyDiscountByCategory("electronica", 10)  
productService.increaseStockByCategory("libros", 50)  
  
productService.upsert(  
    Product(nombre = "Auriculares", precio = 89.99, stock = 60, categoria = "electronica")  
)  
  
productService.removeDiscounts()
```

1.11 4. Módulo 4: proyecto integrado de biblioteca digital

1.11.1 Qué vas a aprender

En este módulo se integran las operaciones anteriores en un dominio más realista: una biblioteca digital.

El proyecto usa tres modelos:

- Autor
- Libro
- Prestamo

Y dos capas sencillas:

- LibraryRepository / MongoLibraryRepository: acceso a MongoDB.
- BibliotecaService: validaciones y operaciones de negocio.

1.11.2 Modelos principales

```
import org.bson.types.ObjectId

data class Autor(
    val _id: String = ObjectId().toHexString(),
    val nombre: String,
    val nacionalidad: String,
    val anioNacimiento: Int,
)
```

```
import org.bson.types.ObjectId

data class Libro(
    val _id: String = ObjectId().toHexString(),
    val titulo: String,
    val autorNombre: String,
    val anio: Int,
    val genero: String,
    val disponible: Boolean = true,
    val copias: Int,
)
```

```
import org.bson.types.ObjectId
data class Prestamo(
    val _id: String = ObjectId().toHexString(),
    val usuario: String,
    val libroTitulo: String,
    val fechaPrestamo: String,
    val devuelto: Boolean = false,
)
```

1.11.3 Crear el servicio de biblioteca

```
val bibliotecaService = BibliotecaService(
    MongoLibraryRepository(
```

```
        autores = database.getCollection<Autor>("autores"),
        libros = database.getCollection<Libro>("libros"),
        prestamos = database.getCollection<Prestamo>("prestamos"),
    )
}
```

1.11.4 Operaciones disponibles

```
bibliotecaService.registerAutores(autores)
bibliotecaService.registerLibros(libros)
bibliotecaService.registerPrestamos(prestamos)

val disponibles = bibliotecaService.availableBooks()
val pendientes = bibliotecaService.pendingLoans()

bibliotecaService.returnLoan(prestamo)
bibliotecaService.updateAvailabilityWithoutCopies()
bibliotecaService.deleteReturnedLoans()

val counts = bibliotecaService.counts()
```

1.12 Ejercicio 5: biblioteca digital completa

1.12.1 Objetivo

Construir una pequeña biblioteca digital con autores, libros y préstamos.

1.12.2 Pasos

1. Crea las colecciones autores, libros y prestamos.
2. Inserta al menos 3 autores.
3. Inserta al menos 6 libros.
4. Inserta 3 préstamos.
5. Consulta los libros disponibles.
6. Consulta los préstamos no devueltos.
7. Marca un préstamo como devuelto y aumenta en 1 las copias del libro.
8. Marca como no disponibles los libros sin copias.
9. Borra los préstamos devueltos.
10. Muestra el recuento final de documentos.

1.12.3 Solución orientativa

```
val autores = listOf(
    Autor(nombre = "Miguel de Cervantes", nacionalidad = "Española", anioNacimiento = 1547),
```

```
Autor(nombre = "Mary Shelley", nacionalidad = "Británica", anioNacimiento = 1797),
Autor(nombre = "Frank Herbert", nacionalidad = "Estadounidense", anioNacimiento = 1920),
)

val libros = listOf(
    Libro(titulo = "El Quijote", autorNombre = "Miguel de Cervantes", anio = 1605, genero = "Novela",
    Libro(titulo = "Frankenstein", autorNombre = "Mary Shelley", anio = 1818, genero = "Ciencia ficción",
    Libro(titulo = "Dune", autorNombre = "Frank Herbert", anio = 1965, genero = "Ciencia ficción",
    Libro(titulo = "Novelas ejemplares", autorNombre = "Miguel de Cervantes", anio = 1613, genero = "Novela",
    Libro(titulo = "El último hombre", autorNombre = "Mary Shelley", anio = 1826, genero = "Ciencia ficción",
    Libro(titulo = "Mesías de Dune", autorNombre = "Frank Herbert", anio = 1969, genero = "Ciencia ficción",
)

val prestamos = listOf(
    Prestamo(usuario = "Ana García", libroTitulo = "El Quijote", fechaPrestamo = "2026-05-08"),
    Prestamo(usuario = "Luis Martín", libroTitulo = "Dune", fechaPrestamo = "2026-05-08"),
    Prestamo(usuario = "María López", libroTitulo = "Frankenstein", fechaPrestamo = "2026-05-08")
)

bibliotecaService.registerAutores(autores)
bibliotecaService.registerLibros(libros)
bibliotecaService.registerPrestamos(prestamos)

println(bibliotecaService.availableBooks())
println(bibliotecaService.pendingLoans())

bibliotecaService.returnLoan(prestamos.first())
bibliotecaService.updateAvailabilityWithoutCopies()
bibliotecaService.deleteReturnedLoans()

println(bibliotecaService.counts())
```

1.13 Pruebas del proyecto

1.13.1 Qué se prueba

Las pruebas unitarias no necesitan MongoDB real. Se centran en:

- Validación de configuración (MongoConfigTest).
- Validación de nombres de bases de datos y colecciones (DatabaseManagerTest).
- Comportamiento de ProductService con ProductRepository simulado.
- Comportamiento de BibliotecaService con LibraryRepository simulado.

1.13.2 Herramientas usadas

- Kotest como framework de pruebas.
- DescribeSpec como estilo de especificación.
- MockK para simular repositorios.

Ejemplo de prueba:

```
class ProductServiceTest : DescribeSpec({
    describe("ProductService") {
        it("should reject a product with negative price") {
            val repository = mockk<ProductRepository>()
            val service = ProductService(repository)

            shouldThrow<IllegalArgumentException> {
                service.register(
                    Product(nombre = "Inválido", precio = -1.0, stock = 1, categoria = "test")
                )
            }
        }
    }
})
```

1.14 Ejecutar el proyecto

1.14.1 Ejecutar las pruebas

```
./gradlew test
```

El proyecto incluye una prueba de integración en:

```
src/test/kotlin/org/iesra/tallermongo/integration/MongoWorkshopIntegrationTest.kt
```

Esta prueba usa una base de datos temporal llamada `taller_mongo_integration_test`, ejecuta operaciones reales contra MongoDB y la elimina al finalizar.

1.14.1.1 Comportamiento si no hay conexión configurada La prueba de integración solo se ejecuta si existe la variable `MONGODB_URI`. Si no está definida, Gradle no falla: la prueba queda marcada como omitida.

En el informe de tests verás algo similar a:

```
tests="1" skipped="1" failures="0" errors="0"
```

Esto significa que la prueba está preparada y compila, pero no se ha ejecutado contra MongoDB real porque no había URI disponible.

1.14.1.2 Ejecutar la prueba de integración contra MongoDB Atlas Para ejecutarla realmente, define la URI de Atlas y lanza los tests con una JVM compatible:

```
export MONGODB_URI="mongodb+srv://usuario:password@cluster.mongodb.net/"
JAVA_HOME="/home/edu/.sdkman/candidates/java/21.0.1-tem" ./gradlew test
```

Si también quieres indicar una base de datos por defecto para la aplicación, puedes definir:

```
export MONGODB_DATABASE="taller_mongo"
```

La prueba de integración no usa esa base de datos por defecto: trabaja con `taller_mongo_integration_test` para no mezclar datos de prueba con datos del taller.

1.14.2 Ejecutar la aplicación

Primero define la configuración:

```
export MONGODB_URI="mongodb+srv://usuario:password@cluster.mongodb.net/"
export MONGODB_DATABASE="taller_mongo"
```

Después ejecuta `Main.kt` desde el IDE o configura una tarea de ejecución Gradle si el proyecto la incorpora más adelante.

1.15 Resumen de clases principales

Clase	Responsabilidad
MongoConfig	Leer y validar configuración externa
MongoConnection	Crear y cerrar el cliente MongoDB
DatabaseManager	Operaciones sobre bases de datos
CollectionManager	Operaciones sobre colecciones
ProductService	Validar y coordinar CRUD de productos
MongoProductRepository	CRUD de productos con KMongo
BibliotecaService	Operaciones del proyecto integrado
MongoLibraryRepository	Persistencia de biblioteca con KMongo

1.16 Buenas prácticas aplicadas

- No hay credenciales en el código fuente.
- Se usan variables de entorno para la conexión.
- Se usan colecciones tipadas con `data class`.
- La lógica se puede probar sin MongoDB real mediante interfaces y `MockK`.
- Las operaciones peligrosas, como eliminar bases de datos, están separadas en métodos explícitos.